

Full-Stack Clothing E-Commerce Development Prompt

You are a senior full-stack software engineer, UI/UX designer, database architect, and security engineer.

Your task is to design and develop a complete, production-quality full-stack clothing e-commerce web application from the ground up.

The application sells:

- Women's clothing, primarily dresses, tops, jeans, trousers, jackets, etc.
- Men's clothing, primarily T-shirts, shirts, jeans, trousers, hoodies, jackets, etc.

The application must have two major experiences:

1. Customer-facing e-commerce storefront
2. Secure admin portal for managing the entire store

Do not build a toy/demo application. Build the application with a clean architecture that can eventually be deployed to production.

1. TECHNOLOGY STACK

Use the following technologies unless there is a strong technical reason to change something:

Frontend:

- Next.js
- React
- TypeScript
- Tailwind CSS
- Modern responsive design
- Server Components where appropriate
- Client Components only where interactivity requires them

Authentication:

- Firebase Authentication
- Google Sign-In
- Email/password authentication
- Firebase UID should be associated with the application's user record

Database:

- Supabase PostgreSQL
- Use PostgreSQL as the application's primary relational database
- Use Prisma ORM or Drizzle ORM
- Prefer Prisma if there is no compelling reason otherwise

Product image storage:

- Cloudflare R2
- Never store actual product images inside PostgreSQL
- PostgreSQL should store image metadata and storage keys/URLs

Payments:

- Stripe will be integrated later
- Design the order/payment architecture so Stripe can be added without restructuring the application

Do NOT implement Stripe initially unless required to make the architecture work.

2. CORE ARCHITECTURE

Use this architecture:

Next.js

|

+---- Firebase Authentication

|

+---- Backend/API

|

+---- Supabase PostgreSQL

|

+---- Cloudflare R2

|

+---- Stripe (future)

Keep business logic on the server.

Never trust client-side data for:

- Prices
- Inventory
- User roles
- Order totals
- Discounts
- Permissions
- Payment status

Validate all important operations server-side.

Use environment variables for all secrets and credentials.

Never expose:

- Database credentials
- Firebase admin credentials
- R2 secret keys
- Other server secrets

to the browser.

3. UI/UX REQUIREMENTS

The frontend should NOT look like a generic Bootstrap/CRUD e-commerce template.

The design should be:

- Simple
- Modern
- Elegant
- Interactive
- Eye-catching
- Fashion-oriented
- Premium-looking

- Fast
- Clean
- Mobile-first
- Responsive

The interface should feel like a modern fashion brand rather than a developer dashboard.

Avoid excessive animations.

Use subtle animations and micro-interactions where they improve the experience.

Examples:

- Smooth hover effects
- Product image transitions
- Animated cart updates
- Smooth dropdowns
- Product card hover states
- Image zoom on product pages
- Smooth filtering
- Loading skeletons
- Toast notifications
- Subtle page transitions
- Animated wishlist heart
- Interactive size/colour selectors

Animations should never make the website feel slow.

4. VISUAL DESIGN

Use a minimal fashion aesthetic.

Prefer:

- Lots of whitespace
- Strong typography
- High-quality product photography
- Large product imagery
- Clean navigation
- Simple colour palette
- Elegant buttons
- Clear hierarchy

Do not use:

- Excessive gradients
- Excessive shadows
- Excessive rounded cards
- Overly colourful UI
- Unnecessary animations
- Cluttered layouts

The website should look good even if there are hundreds of products.

Make the design system reusable.

Create reusable components for:

- Navbar
- Footer
- ProductCard
- ProductGrid
- ProductGallery
- ProductFilters
- Search
- CartDrawer
- CartItem
- SizeSelector
- ColourSelector
- PriceDisplay
- Breadcrumbs
- Modal
- Toast
- Loading states
- Empty states
- Error states
- Pagination

5. CUSTOMER WEBSITE

Create the following pages/routes.

Home:

- /
- Hero section
- Featured collections
- New arrivals
- Best sellers
- Women's collection
- Men's collection
- Promotional section
- Newsletter
- Footer

Shop:

/shop

Features:

- Product grid
- Search
- Filtering
- Sorting
- Pagination or infinite loading

- Responsive layout

Filters:

- Category
- Gender
- Size
- Colour
- Price range
- Availability
- Rating

Sorting:

- Featured
- Newest
- Price low to high
- Price high to low
- Best selling
- Highest rated

Women's:

/women

Categories:

- Dresses
- Tops
- Jeans
- Trousers
- Jackets
- Other

Men's:

/men

Categories:

- T-Shirts
- Shirts
- Jeans
- Trousers
- Hoodies
- Jackets
- Other

Product:

/product/[slug]

Product page must include:

- Large image gallery
- Multiple product images
- Image thumbnails

- Image zoom
- Product name
- Price
- Description
- Available colours
- Available sizes
- Stock availability
- Quantity selector
- Add to cart
- Buy/checkout action placeholder
- Wishlist
- Product details
- Size information
- Reviews section
- Related products

If a variant is unavailable, make that clear visually.

Never allow a customer to select an unavailable variant.

6. SEARCH

Implement a proper product search.

Example:

User searches:

"black dress"

Return relevant products based on:

- Product name
- Description
- Category
- Colour
- Tags

Search should be fast.

Implement debouncing where appropriate.

Create a good search experience on both desktop and mobile.

7. AUTHENTICATION

Use Firebase Authentication.

Support:

- Google Sign-In
- Email/password
- Logout
- Password reset

- Authenticated user state

Create an application user record in PostgreSQL.

Example:

users

- id
- firebase_uid
- email
- name
- avatar_url
- role
- created_at
- updated_at

Roles:

- CUSTOMER
- ADMIN

Never rely only on frontend route protection.

Admin authorization must also be enforced server-side.

8. CUSTOMER ACCOUNT

Create:

/account

Sections:

- Profile
- Orders
- Addresses
- Wishlist
- Account settings

Customers should only be able to access their own information.

9. CART

Implement a proper shopping cart.

Support:

- Add product variant
- Remove item
- Change quantity
- Cart total
- Quantity validation
- Inventory validation
- Empty cart state

A cart item must reference a specific product variant.

For example:

Black T-Shirt / Medium

is different from:

Black T-Shirt / Large

Do not treat the base product as the inventory unit.

10. PRODUCT DATABASE MODEL

Design the PostgreSQL database properly.

At minimum include:

users

categories

products

product_images

product_variants

inventory

carts

cart_items

wishlist

wishlist_items

addresses

orders

order_items

reviews

Products should support:

- Name
- Slug
- Description
- Gender
- Category
- Brand
- Base price
- Status
- Created date
- Updated date

Product variants should support:

- Product
- SKU
- Size
- Colour
- Price override if needed
- Stock

- Status

Example:

Product:

Oversized Cotton T-Shirt

Variants:

Black / S

Black / M

Black / L

Black / XL

White / S

White / M

White / L

White / XL

Inventory must be tracked per variant.

11. PRODUCT IMAGES

Use Cloudflare R2.

Do not store binary image data in PostgreSQL.

Recommended structure:

products/{productId}/

front.webp

back.webp

side.webp

model.webp

detail.webp

Database:

product_images

- id
- product_id
- storage_key
- image_url
- alt_text
- sort_order
- is_primary
- created_at

The application should support multiple images per product.

Admin should be able to:

- Upload images
- Delete images
- Reorder images

- Set primary image
- Add alt text

Prefer direct browser-to-R2 uploads using secure presigned URLs rather than routing large image files through the application server.

12. ADMIN PORTAL

Create a completely separate admin experience.

Route:

/admin

Admin dashboard should be protected.

Only users with ADMIN role may access it.

Dashboard should include:

- Total revenue
- Total orders
- Products sold
- Average order value
- Number of customers
- Low-stock products
- Out-of-stock products
- Recent orders
- Sales chart
- Order status overview

Allow filtering by:

- Today
- Last 7 days
- Last 30 days
- Last 3 months
- Last 12 months
- Custom date range

13. ADMIN PRODUCT MANAGEMENT

Create:

/admin/products

Features:

- Product list
- Search
- Filter
- Create product
- Edit product
- Delete product
- Publish/unpublish

- Manage categories
- Manage variants
- Manage inventory
- Upload product images

Create:

/admin/products/new

The product creation form should allow the admin to define:

- Product name
- Description
- Gender
- Category
- Price
- Sizes
- Colours
- SKU
- Stock
- Images
- Product status

Make the image uploader visually excellent.

Support:

- Drag and drop
- Multiple image upload
- Upload progress
- Image preview
- Reordering
- Primary image selection
- Delete

14. ADMIN INVENTORY

Create:

/admin/inventory

Display inventory at the variant level.

Example:

Oversized T-Shirt

Black:

S - 12

M - 20

L - 8

XL - 4

White:

S - 5

M - 16

L - 11

XL - 2

Highlight:

- Low stock
- Out of stock

Allow authorized admins to update inventory.

Prevent negative stock.

15. ADMIN ORDERS

Create:

/admin/orders

Features:

- View all orders
- Search orders
- Filter by status
- Filter by date
- View order details

Order details:

- Customer
- Products
- Variants
- Quantities
- Prices
- Total
- Shipping address
- Order status
- Payment status
- Created date

Order statuses should support something like:

PENDING

PROCESSING

SHIPPED

DELIVERED

CANCELLED

REFUNDED

Design the database so Stripe can later update payment status using webhooks.

16. SALES ANALYTICS

Admin should be able to see:

- Revenue
- Orders
- Units sold
- Average order value
- Best-selling products
- Best-selling categories
- Revenue over time
- Sales by gender
- Sales by category

Create visually clean charts.

Do not overload the dashboard with charts.

Prioritize useful business information.

17. REVIEWS

Customers should eventually be able to:

- Leave a rating
- Write a review
- View reviews

Prevent users from creating arbitrary duplicate reviews if the business rules don't allow them.

Design review functionality so verified purchases can later be supported.

18. DATABASE SECURITY

Implement proper authorization.

Examples:

Customer:

CAN:

- View public products
- Manage own cart
- Manage own wishlist
- View own orders
- Manage own profile

CANNOT:

- Modify products
- Modify inventory
- View other customers
- Modify other users' orders
- Access admin APIs

Admin:

CAN:

- Manage products

- Manage inventory
- View orders
- Manage order status
- View sales analytics

Use server-side authorization checks for every protected operation.

19. PERFORMANCE

Optimize the application for performance.

Use:

- Next.js image optimization
- Lazy loading
- Responsive images
- WebP/AVIF where appropriate
- Server-side data fetching where appropriate
- Database indexes
- Pagination
- Efficient queries
- Caching where appropriate

Do not load hundreds of products into the browser at once.

Product listings must use pagination or efficient infinite loading.

20. SEO

Implement proper SEO.

Every product should have:

- SEO title
- Meta description
- Canonical URL
- Open Graph metadata
- Structured product information where appropriate

Use clean URLs:

/women/dresses/summer-midi-dress

instead of:

/product?id=123

Generate sitemap and robots configuration.

21. ERROR HANDLING

Implement proper:

- Loading states
- Skeletons

- Empty states
- Error states
- API errors
- Form validation
- Toast notifications

Do not allow uncaught errors to result in broken screens.

22. VALIDATION

Use a validation library such as Zod.

Validate:

- Product creation
- Product updates
- Cart operations
- Address information
- User input
- Query parameters
- API requests

Never trust browser-provided values for price or inventory.

23. ENVIRONMENT VARIABLES

Create:

.env.example

Document all required environment variables.

Examples:

DATABASE_URL=

DIRECT_URL=

Firebase configuration

NEXT_PUBLIC_FIREBASE_API_KEY=

NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=

NEXT_PUBLIC_FIREBASE_PROJECT_ID=

Server-side Firebase credentials must remain private.

R2:

R2_ACCOUNT_ID=

R2_ACCESS_KEY_ID=

R2_SECRET_ACCESS_KEY=

R2_BUCKET_NAME=

R2_PUBLIC_URL=

Do not commit .env files.

24. PROJECT STRUCTURE

Use a clean scalable project structure.

Separate:

- UI components
- Database access
- Authentication
- Business logic
- API routes
- Validation
- Storage
- Utilities
- Types

Do not put the entire application inside a few huge files.

Keep components reusable.

25. CODE QUALITY

Write clean production-quality TypeScript.

Requirements:

- Strong typing
- No unnecessary `any`
- Reusable components
- Clear naming
- Small functions
- Proper error handling
- Comments only where useful
- No duplicated business logic

Do not create fake APIs or fake database responses once the real database layer is implemented.

26. DEVELOPMENT APPROACH

Do NOT try to generate the entire application blindly in one giant response.

Build it in logical milestones.

Milestone 1:

- Initialize project
- Install dependencies
- Configure Next.js
- Configure Tailwind
- Configure Supabase PostgreSQL
- Configure Prisma/Drizzle
- Create database schema
- Create migrations
- Configure Firebase

Milestone 2:

- Authentication
- User model
- Customer authorization
- Admin authorization

Milestone 3:

- Product/category/variant system
- Database queries
- Product APIs/server actions

Milestone 4:

- Cloudflare R2 integration
- Secure image upload
- Product image management

Milestone 5:

- Customer storefront
- Product listing
- Search
- Filtering
- Product detail pages

Milestone 6:

- Cart
- Wishlist
- Customer account
- Orders architecture

Milestone 7:

- Admin dashboard
- Product management
- Inventory
- Orders
- Analytics

Milestone 8:

- Performance
- SEO
- Security audit
- Testing
- Production preparation

Stripe will be added after the above system is stable.

27. IMPORTANT DEVELOPMENT RULE

Before writing significant code, first explain:

1. Proposed architecture
2. Database schema
3. Entity relationships

4. Folder structure
5. Authentication flow
6. Image upload flow
7. Admin authorization flow
8. Development milestones

Then wait for approval before implementing the first milestone.

When implementing each milestone:

- Explain what you are building
- Show the relevant files
- Write complete code
- Explain where each file belongs
- Include installation commands
- Include environment variables
- Include database migrations
- Include testing instructions

Do not skip configuration details.

If you discover an architectural problem, stop and explain it before continuing rather than silently creating technical debt.

The final application should be something that can realistically evolve into a production clothing e-commerce business, not merely a visual prototype.